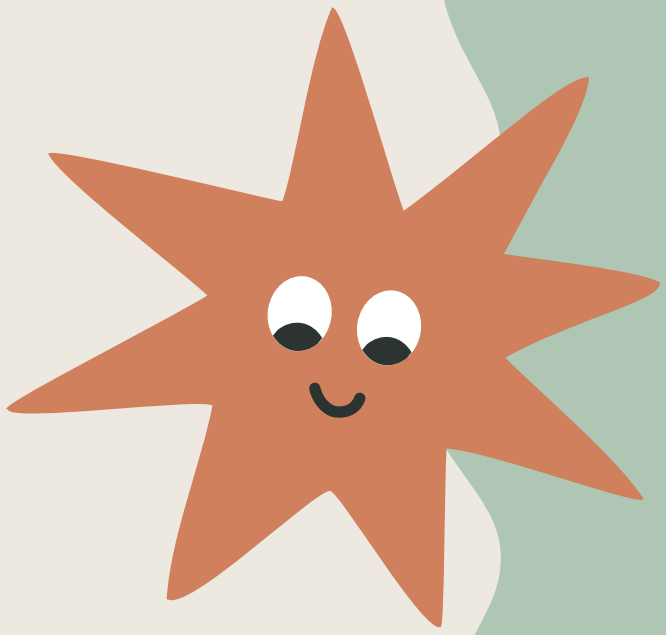
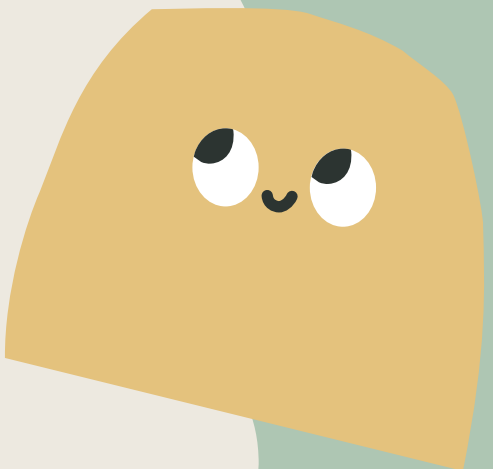
Iterative Deepening Search (IDS) in Artificial Intelligence

Lecture 14





Learning Objectives

➢By the end of this lecture, students should be able to:

- Define Iterative Deepening Search and explain why it is an uninformed search strategy.
- Explain Depth-Limited Search (DLS), which is the foundation of IDS.
- Trace IDS using increasing depth limits.
- Explain why IDS revisits nodes during successive iterations.
- Compare IDS with BFS and DFS.
- Explain the completeness, optimality, time complexity, and space complexity of IDS.
- Identify situations where IDS is useful.

Prepared By: Ehtisham Rasheed

What Is Iterative Deepening Search (IDS)?

- Iterative Deepening Search (IDS) is an uninformed search algorithm that repeatedly performs Depth-Limited Search, increasing the depth limit by one each time until the goal is found or the search space is exhausted
- Instead of exploring the entire graph level by level, IDS performs a depth-first search with a restricted depth
- If the goal is not found within that limit, it starts another search with a larger limit
- The sequence of limits is:

$$0, 1, 2, 3, 4, \ldots$$

Prepared By: Ehtisham Rasheed

Understanding Depth-Limited Search (DLS)

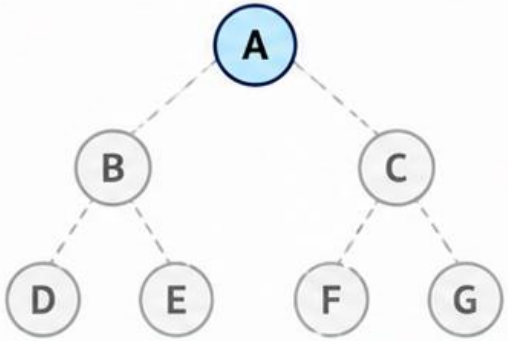
- DLS is a variation of DFS that imposes a maximum depth on the search
- For example, suppose the search starts at node A:
  - At depth 0, only A can be explored
  - At depth 1, A and its immediate children can be explored
  - At depth 2, the search can explore grandchildren as well
  - Nodes deeper than the limit are not expanded during that iteration
- The depth of the starting node is defined as 0

Prepared By: Entisham Rasheed

How Iterative Deepening Search (IDS) Works

Run Depth-Limited Search (DLS) repeatedly with increasing depth limit until the goal is found

Iteration 1: Depth Limit = 0

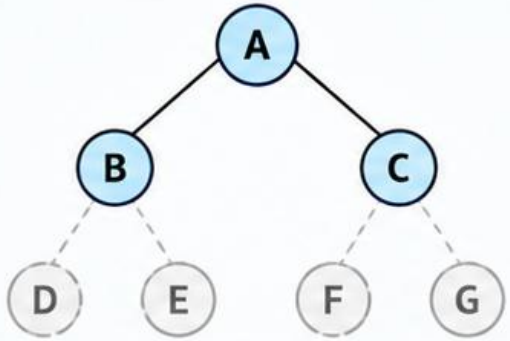


- Start at node **A**.
- Depth limit is 0, so only **A** is visited.
- Goal **F** is not found.
- Increase depth limit to **1**.

Traversal Order (limit = 0)

A

Iteration 2: Depth Limit = 1

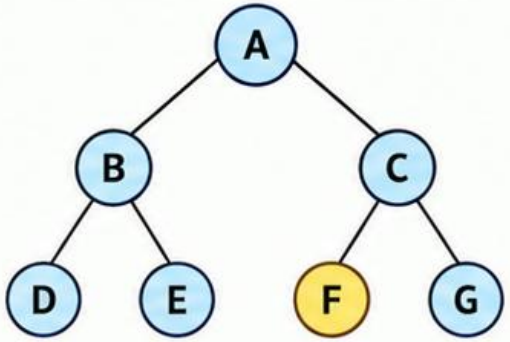


- Start again from **A** (new iteration).
- Depth limit is 1, so **A**, **B** and **C** are visited.
- Goal **F** is at depth 2, so it is not found.
- Increase depth limit to 2.

Traversal Order (limit = 1)

A → B → C

Iteration 3: Depth Limit = 2

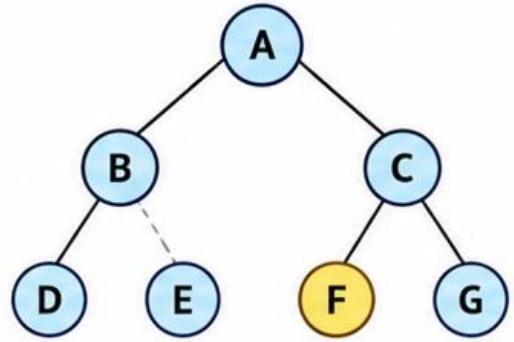


- Start again from **A** (new iteration).
- Depth limit is 2, so nodes up to depth 2 are visited.
- Goal **F** is found at depth **2**.
- Search **stops**.

Traversal Order (limit = 2)

A → B → D → E → C → F

Iteration 4: Not Needed



- Not required because the goal was already found in iteration 3.
- If the goal was not found, we would increase the limit to 3 and repeat.

Traversal Order (limit = 3)

-

Key Points

1. Run Depth-Limited Search (DLS) with limits 0, 1, 2, 3, ...
2. Each iteration starts again from the root.
3. If the goal is not found within the current limit, increase the limit by 1.
4. Nodes at depth greater than the current limit are not expanded.
5. When the goal is found, the search stops and returns the path.

Solution Path

A → **C** → **F**

The solution path from **A** to **F** is:

A → C → F

Understanding the depth limit versus the goal depth

- **Depth limit:** The maximum depth allowed during the current DLS iteration.
- **Goal depth:** The depth at which the goal node actually exists.
- For example, if the goal is at depth 4:

Iteration	Depth limit	Can IDS reach the goal?
1	0	No
2	1	No
3	2	No
4	3	No
5	4	Yes

- If the goal is first found at depth d, IDS performs searches with limits $0,1,\ldots,d$ stopping when the goal is found.

Prep-Isham Rasheed

Why does IDS repeat work?

- BFS usually visits each node once in a graph-search implementation, whereas IDS repeatedly starts from the root with a larger depth limit
- For example:
  - Limit 0 explores A
  - Limit 1 explores A, B and C
  - Limit 2 explores A, B, D, E, C, F and G
- In a tree with a branching factor b = 10, the bottom layer holds roughly **90%** of all nodes. Regenerating upper levels only adds a minor overhead (roughly 11% total extra work)

Prepared By: Ehtisham Rasheed

Property	BFS	DFS	IDS
Strategy	Level by level	Deepest branch first	Repeated depth-limited DFS
Main data structure	Queue	Stack	Stack/DFS recursion per iteration
Complete?	Yes with finite branching	Not always in infinite-depth spaces	Yes with finite branching and a finite-depth solution
Optimal?	Yes for equal step costs	No, in general	Yes for equal step costs
Time complexity	$O(b^{d+1})$	$O(b^m)$	$O(b^d)$
Space complexity	$O(b^{d+1})$	$O(bm)$	$O(bd)$
Repeats shallow nodes?	Usually not in tree search	Not inherently	Yes, by design

IDS Properties

- Is IDS Complete?
  - Yes, IDS is complete on finite graph
- Is IDS Optimal?
  - IDS finds a optimal solution when branching factor is finite and each step has the same cost
- Time and Space Complexity
  - Let b = branching factor and d = depth of the shallowest goal
- IDS has the following commonly taught worst-case complexities:
  - Time Tree = $O(b^d)$ Graph = $O(V + E)$
  - Space Tree = $O(bd)$ Graph = $O(V)$

Prepared By: Ehtisham Rasheed

Advantages and Disadvantages

- Advantages
  - Uses much less memory than BFS in typical search-tree settings
  - Finds a shallowest solution
  - Finds a solution at finite depth when branching is finite
- Disadvantages
  - Shallow nodes are visited multiple times
  - The search tree can grow exponentially
  - A shallow path may be more expensive than a deeper one

Prepared By: Ehtisham Rasheed

Practical AI Applications

- Puzzle Solving
  - Searching for a solution to puzzles where the number of moves is unknown
- Game-Tree Exploration
  - Exploring possible sequences of moves up to increasing depths
- State-Space Search
  - Finding a sequence of actions that reaches a target state
- Planning Problems
  - Searching for a plan with the fewest steps when each action has equal cost
- AI Problem Solving
  - Exploring a large search tree without storing every node at a given level

Prepared By: Ehtisham Rasheed